Why Decision Support Systems Evolve: A Critical Review of Computational Capacity, Architectures, and Decision Requirements

Unattributed review draft

***Abstract*—Decision support systems (DSS) have expanded from interactive analytical models to knowledge-based systems, collaborative platforms, data warehouses, predictive models, and language-model-based assistants and agents. A chronology alone does not explain these transitions. This critical narrative review compares the limitation motivating each transition, the contribution of computational capacity, the architectural innovation introduced, and the decision problem left unresolved. It draws on a repository catalog of 67 records and a purposive selection of accessible foundational and technical papers. The synthesis indicates that computing capacity primarily expands feasible scale, whereas architectures change how systems represent knowledge, integrate evidence, interact with users, and execute actions. Neither mechanism alone establishes decision quality. Explicit preferences, valid objectives, trustworthy data, and organizational accountability remain necessary across generations. Recent retrieval and agent architectures reuse earlier ideas of external knowledge, modular tools, and human oversight rather than eliminating them. A transition matrix and a proposed evaluation framework connect historical developments to practical choices about when to scale, redesign, or retain an existing DSS. The review makes no quantitative causal estimate or claim of exhaustive literature coverage.**

***Index Terms*—decision support systems, architectural evolution, computational capacity, business intelligence, expert systems, large language models, human oversight**

I. Introduction

Decision support is concerned with improving decisions, not simply producing more information. A system may calculate an optimum, retrieve an explanation, predict an outcome, or recommend an action; these are different contributions to a decision process. Historical accounts document the expansion from individual analytical support to group, organizational, and Web-based support [1], [2]. The important question for system design is why a particular expansion became necessary and what made it possible.

Two explanations recur. One attributes progress to cheaper and more powerful computation, larger memories, faster networks, and more accessible data storage. The other emphasizes representations and architectures: separating a knowledge base from inference, separating analytical from transactional workloads, learning representations, retrieving external evidence, or orchestrating tools. The explanations interact. An architecture determines which computations are useful and parallelizable; available hardware determines whether the architecture can operate at acceptable cost and latency.

This review asks: (RQ1) Which limitations motivated movement between major DSS approaches? (RQ2) What distinct roles did computational capacity, architecture, data, and organizational requirements play? (RQ3) Which unresolved limitations explain the direction of subsequent development? Its contribution is a consistent transition comparison rather than a claim that all systems followed a single sequence. Model-driven, knowledge-driven, data-driven, document-driven, and communications-driven DSS coexist in Power's classification [2]. The periods used below describe the emergence and increasing visibility of approaches, not dates when earlier approaches disappeared.

II. Review Design and Evidence Boundaries

A. Corpus and Selection

The starting corpus is the repository's `DSS_Evolution_Literature.json`, containing 67 records spanning 1965–2026. It includes 42 direct PDF links, of which 38 files were obtained during environment preparation. File counts do not establish 38 independent, relevant studies: versions, overlapping topics, and metadata errors require separate screening. This review uses a purposive subset linking historical DSS syntheses to technical contributions that expose a transition mechanism. The selected set covers historical classification, knowledge engineering, preference modeling, data integration, machine learning, explainability, Transformers, retrieval, and tool use.

The 24 cited records are listed in the companion evidence matrix. Twenty-three have locally available PDF text; the PROMETHEE record is supported here by publisher metadata because its downloaded article pages are not usefully text-extractable. Such access limitations are not interpreted as negative research findings. Early developments are also interpreted through historical syntheses, so some historical conclusions rely on secondary evidence. Recent catalog entries with insufficiently checked identities or publication details are not used simply because their filenames appear relevant.

B. Extraction and Synthesis

For each selected work, the synthesis records: the problem described; the proposed mechanism; its computational or infrastructural requirements; and the limitation that persists. Full-text abstracts, introductions, and mechanism discussions were used to check these relationships. References retain catalog identifiers in the evidence matrix for traceability. Bibliographic information is taken from accessible article text

or the catalog and identified conservatively where only a preprint was checked.

This is a critical narrative review, not a systematic review or meta-analysis. No independent database search, preregistered protocol, dual screening, risk-of-bias scoring, or pooled effect estimate was conducted. Inclusion is driven by explanatory relevance and access, introducing selection and availability bias. A paper's benchmark result supports a capability under its reported conditions; it does not establish enterprise adoption or causally explain a field-wide historical transition. Statements connecting different periods are the review's interpretive synthesis.

C. Comparison Framework

Four drivers organize the comparison. *Capacity* concerns feasible data volume, memory, throughput, and latency. *Architecture* concerns representations, interfaces, component boundaries, and control flow. *Data and knowledge* concern usable observations, labels, documents, and expertise. *Decision requirements* concern objectives, preferences, collaboration, accountability, and workflow fit. These categories avoid treating faster hardware as evidence of better decisions or treating every software change as a new generation.

III. Evolution Through Unresolved Bottlenecks

A. Interactive Model-Driven Support: From Reports to Exploration

Early DSS distinguished interactive analytical support from routine information reporting. In the historical synthesis of Shim *et al.*, classic systems combine database capabilities, modeling, and an interface [1]. The motivating limitation is that a fixed report describes a situation but does not necessarily let a manager explore alternatives or change assumptions. Interactive models provide what-if analysis, optimization, and sensitivity analysis while leaving judgment with the decision maker.

Greater computational availability makes repeated calculations and interactive response practical. The architectural contribution is equally important: data access, model execution, and the dialogue with a user must work together. A faster batch report does not automatically become a usable interactive decision model. The resulting systems remain constrained by assumptions, the cost of maintaining models, and their limited representation of tacit domain knowledge. This motivates knowledge-oriented support for tasks whose expertise cannot easily be expressed as one numerical objective. It also motivates collaboration when several stakeholders must agree on objectives. These are branching responses, not the replacement of mathematical models.

B. Knowledge-Based Support: From Equations to Explicit Expertise

Knowledge engineering addresses tasks in which expert judgments and domain-specific inference are central. Feigenbaum discusses acquiring, representing, and using expert knowledge, including explanations of reasoning [3]. The important innovation is an explicit representation of knowledge that an inference process can apply. Later expert-system literature documents a variety of methods and applications rather than a single uniform implementation [4].

More processor time and memory permit larger rule sets and broader search, but do not supply missing expertise. A rule-based architecture can expose the basis for a conclusion, yet its quality depends on the rules, their scope, and their maintenance. The knowledge-acquisition and maintenance problem is therefore representational and organizational as well as computational. Expanding hardware cannot ensure that an expert's exception has been elicited or that a rule remains valid after a policy change.

These limitations help explain the attraction of learning patterns from observations. However, statistical learning changes the source of knowledge rather than abolishing the need for domain expertise. Labels, target definitions, and evaluation criteria still encode expert choices. A subsequent generation may reduce manual rule authoring while increasing demands for representative data and monitoring.

C. Preference and Group Support: From One Objective to Negotiated Choices

Many decision problems have several criteria and several decision makers. Faster optimization does not determine how to trade cost against safety, or whose preference should prevail. Multi-criteria methods, including preference-ranking approaches, provide explicit machinery for comparing alternatives [5]. The modern comparison between multiple-criteria decision aiding and preference learning shows both shared goals and different assumptions about how decision models are constructed [6].

Group and communications support respond to coordination requirements described in the DSS historical literature [1], [2]. Networking increases access and communication speed; collaborative interfaces change participation and information exchange. Neither alone resolves disagreement. A technically efficient group system may reproduce unequal participation or give an unjustified appearance of consensus.

This strand persists throughout the history. Data-driven systems can estimate consequences without choosing acceptable trade-offs. Language models can articulate a preference without having authority to set it. The transition pressure is thus not simply "more compute" but the need to make values, alternatives, and participation explicit. Preference modeling becomes a complement to later predictive and generative systems.

D. Warehousing, OLAP, and Web DSS: From Isolated Models to Shared Data

As organizations accumulate operational records, isolated application databases and manually assembled inputs become obstacles. Chaudhuri and Dayal describe data warehousing and online analytical processing (OLAP) as decision-support

technologies, including the distinct needs of analytical workloads [7]. Warehousing integrates data for analysis; multidimensional structures and query techniques support repeated analytical exploration. Web-based DSS broaden access and organizational reach in the historical account [1].

Larger storage, faster processors, and better networks make integration and interactive queries feasible at greater scale. The decisive architectural change is not merely putting old software on a faster machine. Separating analytical workloads from transaction processing and providing a shared analytical representation changes what questions users can ask without disrupting operations. Web interfaces also change deployment and distribution rather than the underlying validity of a decision model.

The new bottleneck becomes meaning and relevance. Integrating inconsistent identifiers, definitions, and update schedules is a governance task. A dashboard can display correlations without explaining causes or selecting an action. Consequently, warehousing supports subsequent data mining and prediction, but high data volume does not itself guarantee useful knowledge.

E. Data Mining and Distributed Analytics: From Access to Discovery

Fayyad *et al.* distinguish knowledge discovery as a broader process from the algorithmic step of data mining [8]. The motivating limitation is that access to records and aggregates does not automatically reveal useful patterns. Selection, preparation, interpretation, and evaluation matter alongside the mining algorithm.

At larger scale, implementation complexity also becomes a limitation. Dean and Ghemawat's MapReduce combines a programming model with an implementation that handles distribution and failures [9]. Additional machines provide capacity; the abstraction makes some large-scale computations manageable. It illustrates architecture and hardware enabling one another: a cluster's aggregate processing power is not useful if application developers cannot reliably divide and execute the workload.

Distributed processing addresses throughput for suitable tasks, not every interactive requirement. Batch-friendly computation does not necessarily meet low-latency operational decisions, and an efficient pipeline can propagate poor-quality inputs more quickly. The pressure toward predictive systems is therefore coupled with continued needs for data quality, valid targets, and deployment integration.

F. Machine Learning and Deep Learning: From Crafted Rules to Learned Representations

Predictive machine learning learns relationships from observations rather than requiring every decision rule to be written explicitly. Random forests illustrate an ensemble architecture for prediction [10]; Jordan and Mitchell place learning within a broader technological and scientific context [11]. These approaches support ranking, forecasting, and risk estimation, but the model's output is not automatically a decision prescription.

Deep learning extends the shift from hand-designed features toward learned representations. LeCun *et al.* describe multilayer models and their ability to learn structure in images, speech, and other data [12]. Increased computation and available training data help make large models practical. The architectural contribution is the representation and training procedure that can exploit those resources. More hardware applied to an unsuitable representation is not equivalent to a more effective learning system.

The new limitations include opaque predictions, changing input distributions, and mismatch between predictive loss and decision costs. Ribeiro *et al.* and Lundberg and Lee address explaining model predictions [13], [14]. An explanation can help inspect model behavior; it does not prove causal validity, fairness, or reliability under a distribution shift. The next design pressure is therefore to connect learned outputs with human understanding and decision accountability, rather than simply maximizing predictive accuracy.

G. Transformers and Foundation Models: From Task Models to Reusable Interfaces

Task-specific machine-learning systems often require separate pipelines, labels, and interfaces. The Transformer replaces recurrence with attention-based sequence modeling and supports more parallelizable training in the reported translation setting [15]. This is a concrete case where an architectural choice changes how effectively available hardware can be used. It should not be read as showing that compute alone caused the later adoption of language models in DSS.

Large pretrained language models support reuse across tasks. Brown *et al.* demonstrate few-shot behavior for a large language model [16]. For decision support, natural-language interaction can reduce the effort needed to query documents, describe alternatives, and produce summaries. Chain-of-thought prompting further explores eliciting intermediate reasoning in particular benchmark settings [17].

These developments do not establish that fluent explanations are valid decisions. A generated answer can be unsupported, outdated, inconsistent with policy, or insensitive to stakeholder preferences. Larger models may improve some capabilities while increasing training and serving costs. The resulting pressure is toward external evidence, explicit tools, and checks that do not depend solely on the model's own generated text.

H. Retrieval-Augmented Support: From Parametric Memory to Inspectable Evidence

Lewis *et al.* identify limitations in accessing knowledge, updating it, and providing provenance, and combine a generator with non-parametric retrieval [18]. Retrieval-augmented generation (RAG) changes the information path: an external collection supplies evidence at inference time. It is therefore a response to a knowledge-access and maintenance problem, not simply another increase in model size.

Storage and search capacity support indexing larger collections; the architecture couples retrieval to generation. In DSS,

this separation offers a way to use current organizational documents while keeping document updates distinct from model training. Nevertheless, retrieval can miss relevant material, retrieve conflicting evidence, or expose documents to users who lack access. A citation does not establish that the cited passage supports the generated claim. Evaluation must distinguish retrieval coverage, evidence relevance, answer support, and decision usefulness.

RAG also reveals continuity with earlier DSS: external knowledge stores and user-facing explanations remain central. The implementation differs from symbolic inference, but the need to maintain authoritative knowledge does not disappear.

I. Tool-Using Agents: From Answers to Controlled Actions

An assistant that produces an answer may still leave users to gather additional evidence, calculate values, and execute a workflow. ReAct interleaves reasoning and actions with observations [19]; Toolformer investigates learning when and how to call tools [20]. AutoGen explores multi-agent conversation as an application framework [21]. These mechanisms expand control flow beyond a single model response.

Available inference capacity makes repeated model calls and orchestration feasible. The architectural change is the feedback loop among a model, tools, state, and the environment. Tool interfaces can delegate arithmetic or retrieval to specialized components. Multiple agents can assign different roles, but additional roles do not guarantee independent evidence or reliable agreement. Self-refinement similarly explores iterative feedback [22]; repeated generation should not be equated with external validation.

The new exposure is that an incorrect output can become an incorrect action. Valmeekam *et al.*'s evaluated planning models show substantial limitations in executable planning [23]. Those results concern the models and tasks studied, not every subsequent model. They nevertheless justify evaluating plan validity and execution outcomes separately from linguistic plausibility. An agentic DSS therefore needs explicit action permissions, stopping conditions, observable state, and escalation to an accountable person. Human oversight is a continuing design requirement, not a temporary defect to be removed by scaling.

IV. Cross-Generation Comparison

A. Capacity Expands Feasibility; Architecture Changes the Task

Table I highlights different kinds of change. Faster hardware can shorten optimization time or expand a dataset without changing a system's knowledge representation. A warehouse, by contrast, changes analytical data organization; RAG changes access to external evidence; a tool loop changes whether a system only describes or also acts. These distinctions matter when diagnosing an inadequate DSS. More hardware is appropriate when a suitable computation cannot meet workload requirements. Redesign is necessary when the system cannot represent a requirement or obtain the needed evidence.

The interaction is visible in MapReduce and the Transformer: both organize computation so available resources can be used differently [9], [15]. Their demonstrated settings should not be generalized into a universal claim that distributed processing or attention is best for every DSS. A small, auditable optimization model can remain more appropriate than a large generative system when the decision structure and constraints are explicit.

B. Data Availability Is Not Decision Sufficiency

The path from rules to learned models shifts effort from eliciting explicit knowledge to collecting observations and defining learning objectives. The shift from parametric language models to RAG moves some knowledge maintenance into a document pipeline. In each case, a burden is relocated rather than removed. Data completeness, consent, provenance, and relevance remain prerequisites. Prediction quality also differs from decision quality: an accurate demand forecast may still lead to a poor inventory decision if shortage costs or constraints are wrong.

Preference learning and multi-criteria decision aiding offer a useful bridge because they connect learned evidence with explicit comparisons and values [6]. However, inferred behavior should not automatically be treated as an endorsed preference. Hybrid systems should let users inspect or revise consequential objectives rather than silently deriving authority from historical data.

C. No Single Replacement Sequence

The comparison does not imply that expert systems replaced models, that learning replaced explicit rules, or that agents replaced business intelligence. Power's classification separates the dominant technology from deployment and target users [2]. A modern architecture may combine a warehouse, predictive model, constrained optimizer, policy rules, and a conversational interface. The broader DSS literature also cautions against neglecting practical relevance and decision-theoretic foundations [24].

The most defensible evolutionary interpretation is accumulation and recombination. New components address constraints that older components leave unresolved, but earlier components can remain authoritative within their scope. In particular, an LLM can gather or explain evidence while a validated numerical solver performs optimization and a human approves a consequential action.

V. Evaluation Implications and Research Agenda

The following framework is proposed by this review; it has not been experimentally validated. Evaluate a DSS at five levels. First, test evidence quality: coverage, freshness, provenance, and access restrictions. Second, test model or rule validity on task-relevant cases and changes in operating conditions. Third, test decision quality using explicit utilities, constraints, and sensitivity to preferences. Fourth, test interaction and workflow effects, including user understanding, correction, and escalation. Fifth, measure total operational

TABLE I
TRANSITION MECHANISMS: INTERPRETIVE SYNTHESIS OF THE CITED LITERATURE. PERIODS OVERLAP AND DO NOT IMPLY UNIVERSAL REPLACEMENT.

Approach and emergence	Prior bottleneck / demand	Capacity contribution	Architectural contribution	Remaining pressure
Interactive model DSS (1960s–1970s)	Fixed reporting; semi-structured choices	Repeated calculations at usable latency	Data–model–dialogue integration	Model assumptions; tacit expertise [1], [2]
Knowledge-based DSS (1970s–1980s)	Need to operationalize domain expertise	More rules and inference search	Explicit knowledge and inference	Acquisition, maintenance, narrow scope [3], [4]
Preference/group DSS (1970s onward)	Conflicting criteria and stakeholders	Interactive comparison; networking	Preference models and participation mechanisms	Unresolved value judgments [1], [5], [6]
Warehouse/Web DSS (1990s–2000s)	Fragmented data and restricted access	Storage, query speed, connectivity	Analytical integration and shared interfaces	Semantics and useful interpretation [1], [7]
Mining/distributed analytics (1990s–2000s)	Aggregates insufficient; data too large	Cluster throughput	Discovery pipeline; fault-tolerant execution	Data quality; operational latency [8], [9]
ML/deep learning (2000s–2010s)	Hand-crafted rules/features	Training and inference capacity	Ensembles; learned representations	Opacity, drift, objective mismatch [10]–[14]
Foundation-model assistants (2017 onward)	Task-specific interfaces and adaptation	Large-scale pretraining and serving	Attention; reusable language interface	Unsupported or stale answers [15]–[17]
Retrieval augmentation (2020 onward)	Knowledge updates and provenance	Indexing and retrieval at scale	External evidence coupled to generation	Retrieval errors; unsupported citations [18]
Tool-using agents (2022 onward)	Answers insufficient for workflow execution	Multiple inference/tool steps	Observe–act loops; orchestration	Invalid actions, cost, accountability [19]–[23]

cost, latency, and action failures. Report these separately instead of combining them into an unexplained score.

To investigate whether a transition is driven by capacity or architecture, compare three configurations where feasible: a baseline; the same design with greater computational resources; and a redesigned system under a matched resource budget. Keep task definitions, data access, and human involvement explicit. For agent systems, include a single-agent baseline, tool-only alternatives, and executable-outcome checks. For RAG, report retrieval and grounding performance separately. Such studies would provide stronger evidence than attributing progress to model size or a new architectural label.

Three research questions follow. How can DSS distinguish disagreement about facts from disagreement about values? When does added orchestration improve outcomes enough to justify latency and failure exposure? How can longitudinal evaluation detect when the benefits of a new representation are offset by knowledge maintenance or organizational costs? These questions connect recent language-model architectures with enduring concerns in knowledge engineering and DSS practice.

VI. LIMITATIONS

This review is limited by its repository-based, purposive corpus and partial extraction rather than exhaustive full-document assessment. It does not reconstruct a complete history of operations research, databases, human–computer interaction, or organizational decision theory. The period labels are heuristic. No adoption dataset or controlled historical comparison establishes the relative causal contribution of hardware, algorithms, and institutions. Technical benchmark papers often concern language or prediction tasks rather than deployed organizational decisions. The review therefore treats their relevance to DSS as a reasoned architectural connection, not direct proof of decision improvement. Recent work also changes rapidly; the catalog's 2026 coverage is not a verified state-of-the-art frontier. Publication-specific IEEE requirements and final author declarations require review before submission.

VII. CONCLUSION

DSS evolve when existing representations, evidence flows, or interaction arrangements no longer meet decision requirements and new computational resources make alternative designs feasible. Computing capacity enables larger and faster workloads; architectures determine how that capacity contributes to knowledge, prediction, interaction, or action. The history is consequently an overlap of model-based reasoning, explicit expertise, preference modeling, shared data, learning, and orchestration. Recent assistants and agents extend this history while retaining its central unresolved problem: connecting technically capable systems to valid, accountable decisions. Selecting the next architecture should begin with the current bottleneck and an explicit decision objective, followed by evidence that the change improves outcomes under realistic resource and governance constraints.

DRAFT AND AI ASSISTANCE DISCLOSURE

This unattributed manuscript was drafted with AI assistance using the accompanying repository corpus. It has not undergone independent scholarly peer review. The submitting author should verify every reference and claim, supply authorship and affiliation information, and adapt the disclosure and formatting to the selected IEEE venue's current requirements. No new experiments or human-subject study are reported.

REFERENCES

[1] J. P. Shim, M. Warkentin, J. F. Courtney, D. J. Power, R. Sharda, and C. Carlsson, "Past, Present, and Future of Decision Support Technology," Decision Support Systems, vol. 33, pp. 111–126, 2002. [Online]. Available: https://scholarworks.uni.edu/facpub/3462/

[2] D. J. Power, "Specifying an Expanded Framework for Classifying and Describing Decision Support Systems," Communications of the Association for Information Systems, vol. 13, art. 13, pp. 158–166, 2004, doi: 10.17705/1CAIS.01313. [Online]. Available: https://aisel.aisnet.org/cais/vol13/iss1/13/

[3] E. A. Feigenbaum, "The Art of Artificial Intelligence: Themes and Case Studies of Knowledge Engineering," in Proc. IJCAI, 1977. [Online]. Available: https://www.ijcai.org/proceedings/1977/

[4] S.-H. Liao, "Expert System Methodologies and Applications: A Decade Review from 1995 to 2004," Expert Systems with Applications, vol. 28, pp. 93–103, 2005, doi: 10.1016/j.eswa.2004.08.003. [Online]. Available: https://doi.org/10.1016/j.eswa.2004.08.003

[5] J.-P. Brans and P. Vincke, "Note—A Preference Ranking Organisation Method," Management Science, vol. 31, no. 6, pp. 647–656, 1985, doi: 10.1287/mnsc.31.6.647. [Online]. Available: https://doi.org/10.1287/mnsc.31.6.647

[6] E. Hüllermeier and R. Słowiński, "Preference Learning and Multiple Criteria Decision Aiding: Differences, Commonalities, and Synergies—Part I," 4OR, vol. 22, pp. 179–209, 2024, doi: 10.1007/s10288-023-00560-6. [Online]. Available: https://link.springer.com/article/10.1007/s10288-023-00560-6

[7] S. Chaudhuri and U. Dayal, "An Overview of Data Warehousing and OLAP Technology," ACM SIGMOD Record, vol. 26, no. 1, pp. 65–74, 1997. [Online]. Available: https://sigmodrecord.org/1997/03/15/an-overview-of-data-warehousing-and-olap-technology/

[8] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth, "From Data Mining to Knowledge Discovery in Databases," AI Magazine, vol. 17, no. 3, pp. 37–54, 1996. [Online]. Available: https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/1230

[9] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," in Proc. OSDI, 2004, pp. 137–150. [Online]. Available: https://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/

[10] L. Breiman, "Random Forests," Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324. [Online]. Available: https://link.springer.com/article/10.1023/A:1010933404324

[11] M. I. Jordan and T. M. Mitchell, "Machine Learning: Trends, Perspectives, and Prospects," Science, vol. 349, no. 6245, pp. 255–260, 2015, doi: 10.1126/science.aaa8415. [Online]. Available: https://doi.org/10.1126/science.aaa8415

[12] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, vol. 521, pp. 436–444, 2015, doi: 10.1038/nature14539. [Online]. Available: https://doi.org/10.1038/nature14539

[13] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You? Explaining the Predictions of Any Classifier," arXiv:1602.04938, 2016. [Online]. Available: https://arxiv.org/abs/1602.04938

[14] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," arXiv:1705.07874, 2017. [Online]. Available: https://arxiv.org/abs/1705.07874

[15] A. Vaswani et al., "Attention Is All You Need," arXiv:1706.03762, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[16] T. B. Brown et al., "Language Models Are Few-Shot Learners," arXiv:2005.14165, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[17] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," arXiv:2201.11903, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[18] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," arXiv:2005.11401, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[19] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023, arXiv:2210.03629. [Online]. Available: https://arxiv.org/abs/2210.03629

[20] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," arXiv:2302.04761, 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[21] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[22] A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," arXiv:2303.17651, 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[23] K. Valmeekam, S. Sreedharan, M. Marquez, A. Olmo, and S. Kambhampati, "On the Planning Abilities of Large Language Models (A Critical Investigation with a Proposed Benchmark)," arXiv:2302.06706, 2023. [Online]. Available: https://arxiv.org/abs/2302.06706

[24] D. Arnott and G. Pervan, "Eight Key Issues for the Decision Support Systems Discipline," Decision Support Systems, vol. 44, pp. 657–672, 2008. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167923607001698